

Abstract Algebra and Number theory foundations of Cryptography.

Q1 Prove Fermat's Little Theorem from group theory's first principles, then explain how it motivates the Miller Rabin probability primality test. Apply the test (base $a=2$) to $n=561$ and explain why 561 is a Carmichael number that fools the simpler Fermat test.

Ans: Fermat's Little Theorem: If ' p ' is a prime number and ' a ' is a positive integer not divisible by ' p ' then,

$$a^{p-1} \equiv 1 \pmod{p}$$

Proof from group theory's first principle:

Consider the multiplicative group,

$$\mathbb{Z}_p^* = \{1, 2, \dots, p-1\}$$

Since p is prime, every non-zero element modulo p has a multiplicative inverse. Therefore, $\mathbb{Z}_p^*$ is a group of order, $|\mathbb{Z}_p^*| = p-1$

Take any $a \in \mathbb{Z}_p^*$. Multiplication by a permutes the elements: $1, 2, \dots, p-1$

Therefore,

$$a \cdot 1, a \cdot 2, \dots, a \cdot (p-1)$$

Hence,

$$(a \cdot 1)(a \cdot 2) \dots (a \cdot (p-1)) \equiv 1 \cdot 2 \dots (p-1) \pmod{p}.$$

Thus, $a^{p-1}(p-1) \equiv (p-1)! \pmod{p}$

Since none of $1, 2, \dots, p-1$ is divisible by p , $(p-1)!$ has an inverse modulo p .

So, $a^{p-1} \equiv 1 \pmod{p}$

(Proved)

FLT motivates Miller Rabin:

For a prime p , FLT tells us that,

$$a^{p-1} \equiv 1 \pmod{p}$$

for every a relatively prime to p .

So, for an unknown number n , we can calculate

$$a^{n-1} \pmod{n} \quad \text{if } a^{n-1} \not\equiv 1 \pmod{n},$$

then n is definitely composite.

In Miller-Rabin primality test we follow —

- ① Find $n-1 = 2^k \times m$
- ② Choose 'a' such that $1 < a < n-1$
- ③ Compute $b_0 = a^m \pmod{n}$, $\dots$, $b_i = b_{i-1}^2 \pmod{n}$.
 - $+1 \rightarrow$ composite
 - $-1 \rightarrow$ probably prime.

P.T.O

$$b_3 = b_2^2 \pmod{n} = 67^2 \pmod{561}$$

$$b_3 = 1 \rightarrow \text{composite}$$

$\therefore 561$ is composite.

561 fools the simple Fermat test because:

The number 561 has the factorization,

$$561 = 3 \times 11 \times 17$$

It is a carmichael number.

A carmichael number is composite but satisfies,

$$a^{n-1} \equiv 1 \pmod{n}$$

for every a such that $\gcd(a, n) = 1$.

$$\text{for } 561, \quad 2^{560} \equiv 1 \pmod{561}$$

Therefore, the simple Fermat test with base 2 incorrectly says that 561 may be prime.

561 is called a carmichael number because it is composite but can still satisfy $a^{560} \equiv 1 \pmod{561}$ for every relatively prime to 561.

Q2 Prove that $\mathbb{Z}$ under addition and multiplication modulo n , is a field if and only if n is prime, Illustrate your proof by contrasting the multiplicative structure of $\mathbb{Z}_{11}$ (prime modulus) with $\mathbb{Z}_{12}$ (composite modulus).

Ans: Theorem: The set ' $\mathbb{Z}_n$ ', under addition and multiplication modulo n is a field if and only if n is prime.

Proof:

• If n is prime:

Let $n = p$, where p is prime

The elements are $\mathbb{Z}_p = \{0, 1, 2, \dots, p-1\}$.

For every non zero element a ,

$$1 \leq a \leq p-1$$

since p is prime, $\gcd(a, p) = 1$.

Therefore, a has a multiplicative inverse modulo p .
Thus every non-zero element has an inverse.

Hence, $\mathbb{Z}_p$ is a field.

P.T.O

If $\mathbb{Z}_n$ is a field:

Suppose n is composite.

Then, $n = ab$

where $1 < a < n, 1 < b < n$

In $\mathbb{Z}_n$, $a \neq 0, b \neq 0$

but, $ab = n \equiv 0 \pmod{n}$

Thus there are non-zero elements whose product is zero.

These are called zero divisors.

A field cannot contain zero divisors.

Therefore, $\mathbb{Z}_n$ can not be a field if n is composite.

Hence, $\mathbb{Z}_n$ is a field iff n is prime.

(Proved)

Example:

In $\mathbb{Z}_{12}$,

$$3 \times 4 = 12 \equiv 0 \pmod{12}$$

but $3 \neq 0, 4 \neq 0$

Therefore 3 and 4 are zero divisors

$\therefore \mathbb{Z}_{12}$ not a field.

P.T.O

On the other hand,

In $\mathbb{Z}_{11}$, every non-zero element has an inverse.

For example, $3 \times 4 = 12 \equiv 1 \pmod{11}$.

So, $3^{-1} \equiv 4 \pmod{11}$.

$\therefore \mathbb{Z}_{11}$ is a field.

$\therefore \mathbb{Z}_n$ is a field exactly when n is prime.

Q3 // Construct $GF(2^3)$ using the irreducible polynomial $x^3 + x + 1$ over $GF(2)$. Build the Multiplication table for a generator element α and determine whether the multiplicative group $GF(2^3)^*$ is cyclic.

Ans: We construct,

$$GF(2^3) = GF(8)$$

over $GF(2) = \{0, 1\}$ using the irreducible polynomial.

$$x^3 + x + 1.$$

Let, α be a root. Then, $\alpha^3 + \alpha + 1 = 0$

Since arithmetic is over $GF(2)$,

$$-1 = 1.$$

Therefore, $\alpha^3 = \alpha + 1$

P.T.O

Elements of $GF(8)$,

$$GF(8) = \{0, 1, \alpha, \alpha^2, \alpha+1, \alpha^2+\alpha, \alpha^2+1, \alpha^2+\alpha+1\}$$

Powers of α ,

$$\alpha^0 = 1$$

$$\alpha^1 = \alpha$$

$$\alpha^2 = \alpha^2$$

$$\alpha^3 = \alpha+1$$

$$\alpha^4 = \alpha^2+\alpha$$

$$\alpha^5 = \alpha(\alpha^2+\alpha) = \alpha^3+\alpha^2 = \alpha+1+\alpha^2 = \alpha^2+\alpha+1$$

$$\alpha^6 = \alpha(\alpha^2+\alpha+1) = \alpha^3+\alpha^2+\alpha$$

$$= \alpha+1+\alpha^2+\alpha = \alpha^2+1$$

$$\alpha^7 = \alpha(\alpha^2+1) = \alpha^3+\alpha$$

$$= \alpha+1+\alpha = 1$$

Thus, $\alpha^7 = 1$

and no smaller positive power equals 1, therefore,

$$\text{ord}(\alpha) = 7$$

Multiplication table:

Since, $a^i a^j = a^{i+j}$ and $a^7 = 1$

$\times$	1	α	α^2	α^3	α^4	α^5	α^6
1	1	α	α^2	α^3	α^4	α^5	α^6
α	α	α^2	α^3	α^4	α^5	α^6	$\alpha^7 = 1$
α^2	α^2	α^3	α^4	α^5	α^6	1	α
α^3	α^3	α^4	α^5	α^6	1	α	α^2
α^4	α^4	α^5	α^6	1	α	α^2	α^3
α^5	α^5	α^6	1	α	α^2	α^3	α^4
α^6	α^6	1	α	α^2	α^3	α^4	α^5

The multiplicative group is,

$$GF(8)^* = GF(8) - \{0\}$$

and has 7 elements,

Since α has order 7,

$GF(8)^*$ is cyclic with generator α .

(Am)

Q4 Derive the time complexity of modular exponentiation via square and multiply, compute $7^{560} \bmod 561$ step by step, and interpret the result in light of 561 being a Carmichael number.

Ans: Square-and-Multiply Complexity:

In square and multiply, the exponent e is exposed bit by bit/processed bit by bit. For each bit.

- 1 squaring is performed
- If the bit is 1, 1 multiplication is performed

The number of bits in e is: $\lfloor \log_2 e \rfloor + 1$

$\therefore$ The time complexity is: $O(\log e)$

Compute $7^{560} \bmod 561$:

At first $560 = 512 + 32 + 16$

$$\text{So, } 7^{560} = 7^{512} \cdot 7^{32} \cdot 7^{16}$$

Now, Successive squaring modulo 561,

$$7^2 \equiv 49 \pmod{561}$$

$$7^4 = 49^2 = 2401 \equiv 157 \pmod{561}$$

$$7^8 = (157)^2 = 24649 \equiv 526 \pmod{561}$$

$$7^{16} = (526)^2 = 276676 \equiv 103 \pmod{561}$$

$$7^{32} = (103)^2 = 10609 \equiv 511 \pmod{561}$$

$$7^{64} = (511)^2 = 261121 \equiv 256 \pmod{561}$$

$$7^{128} = (256)^2 = 65536 \equiv 460 \pmod{561}$$

$$7^{256} = (460)^2 = 211600 \equiv 103 \pmod{561}$$

$$7^{512} = (103)^2 = 10609 \equiv 511 \pmod{561}$$

$$\therefore 7^{560} = 7^{512} \cdot 7^{32} \cdot 7^{16} \equiv 511 \times 511 \times 103 \pmod{561}$$

$$7^{560} \equiv 1 \pmod{561}$$

Interpretation of the result:

$$561 = 3 \times 11 \times 17$$

So, 561 is composite but it is a Carmichael number.

$$\text{Since } \gcd(7, 561) = 1$$

it satisfies, $a^{560} \equiv 1 \pmod{561}$
for every a relatively prime to 561.

Thus the Fermat primality test using base 7 would incorrectly suggest that 561 is prime.

Q5: Compare deterministic (AKS) and probabilistic (Miller-Rabin) primality testing in terms of asymptotic complexity, error probability and practical relevance to RSA key generation at 2048 bit and 4096-bit key sizes.

Ans: Comparison of AKS and Miller-Rabin are given below:

Feature	AKS	Miller-Rabin
Type	Deterministic	Probabilistic.
Correctness	Always correct	Very high probability.
Error	No error	Possible false positive.
Basic idea	Polynomial congruence test	Repeated modular exponentiation.
Practical speed.	Relatively slower	Very fast.
Complexity	Polynomial in " $\log n$ "	Roughly $O(k^3)$ bit operations per round for k -bit integers with schoolbook arithmetic
RSA use	Mainly theoretical	Widely used in practice.

P.T.O

Feature	AKS	Miller Rabin
2048/4096 bit RSA key generation	Not normally used for routine generation	Standard practical choice.

Modular Arithmetic & the Chinese Remainder

Theorem:

Q6: Derive the Extended Euclidean Algorithm formally (as a sequence of linear combinations), then use it to compute $\gcd(240, 46)$ together with the Bézout co-efficients x, y such that $240x + 46y = \gcd(240, 46)$

Ans: Extended Euclidean Algorithm:

The Euclidean Algorithm is based on repeated division:

$$a = q_1 b + r_1$$

$$b = q_2 r_1 + r_2$$

and so on, until the remainder becomes zero. The last non-zero remainder is $\gcd(a, b)$.

The Extended Euclidean Algorithm expresses the $\gcd$ as a linear combination of a and b :

$$ax + by = \gcd(a, b)$$

where x and y are called the Bézout co-efficients.

Now compute $\gcd(240, 46)$ where $A = 240$ and $B = 46$ with extended Euclidean algorithm in a table:

Q	A	B	R	S_1	S_2	S	T_1	T_2	T
5	240	46	10	1	0	1	0	1	-5
4	46	10	6	0	1	-4	1	-5	21
1	10	6	4	1	-4	5	-5	21	-26
1	6	4	2	-4	5	-9	21	-26	47
1	4	2	0	5	-9	23	-26	47	-120
	2	0		-9	23		47	-120	
	$\downarrow$ gcd			$\downarrow$ x			$\downarrow$ y		

Hence, $x = -9$, $y = 47$ [Bezout co-efficients]

$$\therefore ax + by = 240x(-9) + (47)46$$

$$= 2$$

$$\therefore \gcd(a, b) = 2 \quad (\text{Ans})$$

Q7 // State and prove that Chinese Remainder Theorem (existence and uniqueness) for pairwise coprime moduli. Apply it to solve: $x \equiv 2 \pmod{3}$, $x \equiv 3 \pmod{5}$, $x \equiv 4 \pmod{7}$, $x \equiv 5 \pmod{11}$.

Ans: Chinese Remainder Theorem Statement:

Let $m_1, m_2, m_3, \dots, m_n$ be pairwise coprime positive integers. i.e.,

$$\gcd(m_i, m_j) = 1 \quad (i \neq j)$$

Then the system of simultaneous linear congruence,

$$x \equiv a_1 \pmod{m_1}$$

$$x \equiv a_2 \pmod{m_2}$$

$$x \equiv a_n \pmod{m_n}$$

has a solution, and that solution is unique modulo,

$$M = m_1 m_2 \dots m_n$$

Proof of existence:

Let, $M = m_1 m_2 \dots m_n$ and

$$M_i = \frac{M}{m_i}$$

Since the moduli are pairwise co-prime $\gcd(M_i, m_i) = 1$

Therefore there exists y_i such that,

$$M_i y_i \equiv 1 \pmod{m_i}$$

Define the sum as $x = \sum_{i=1}^n a_i M_i y_i$

For modulo m_i , all terms except the i th are divisible by m_i , hence,

$$x \equiv a_i M_i y_i \equiv a_i \pmod{m_i}$$

Therefore, a solution exists.

Proof of uniqueness:

Suppose x and x' are two solutions. Then,

$$x \equiv x' \pmod{m_i} \text{ for every } i.$$

Thus every m_i divides $x - x'$. Since the moduli are pairwise co-prime,

$$M = m_1 m_2 \dots m_n \mid (x - x')$$

Therefore, $x \equiv x' \pmod{M}$

Here is the solution of unique modulo M .

$$\frac{M}{m_i} = M_i$$

Now calculation,

Given that, $x \equiv 2 \pmod{3}$

$x \equiv 3 \pmod{5}$

$x \equiv 4 \pmod{7}$

$x \equiv 5 \pmod{11}$

Comparing to, $x \equiv a \pmod{m}$

We get,

$$a_1 = 2 \quad m_1 = 3$$

$$a_2 = 3 \quad m_2 = 5$$

$$a_3 = 4 \quad m_3 = 7$$

$$a_4 = 5 \quad m_4 = 11$$

$$\therefore M = 3 \times 5 \times 7 \times 11$$

$$M = 1155$$

$$M_1 = 1155/3 = 385$$

$$M_2 = 1155/5 = 231$$

$$M_3 = 1155/7 = 165$$

$$M_4 = 1155/11 = 105$$

$$\text{Now, } M_1 = \frac{M}{m_1} = \frac{1155}{3} = 385$$

$$M_2 = \frac{M}{m_2} = \frac{1155}{5} = 231$$

$$M_3 = \frac{M}{m_3} = \frac{1155}{7} = 165$$

$$M_4 = \frac{M}{m_4} = \frac{1155}{11} = 105$$

Now finds inverse;

$M_1 \times M_1^{-1} = 1 \pmod{m_1}$	$M_2 \times M_2^{-1} = 1 \pmod{m_2}$	$M_3 \times M_3^{-1} = 1 \pmod{m_3}$	$M_4 \times M_4^{-1} = 1 \pmod{m_4}$
$385 \times M_1^{-1} = 1 \pmod{3}$	$231 \times M_2^{-1} = 1 \pmod{5}$	$165 \times M_3^{-1} = 1 \pmod{7}$	$105 \times M_4^{-1} = 1 \pmod{11}$
$385 \times 1 = 1 \pmod{3}$	$231 \times 1 = 1 \pmod{5}$	$165 \times 2 = 1 \pmod{7}$	$105 \times 2 = 1 \pmod{11}$
$\therefore M_1^{-1} = 1$	$\therefore M_2^{-1} = 1$	$\therefore M_3^{-1} = 2$	$\therefore M_4^{-1} = 2$

$$\begin{aligned}
 \therefore X &\equiv (a_1 M_1 M_1^{-1} + a_2 M_2 M_2^{-1} + a_3 M_3 M_3^{-1} + a_4 M_4 M_4^{-1}) \\
 &\equiv (2 \times 385 \times 1 + 3 \times 231 \times 1 + 4 \times 165 \times 2 + 5 \times 105 \times 2) \\
 &\equiv 3833 \pmod{1155} \\
 &\equiv 368 \pmod{1155}
 \end{aligned}$$

Now check,

$$368 \pmod{3} = 2$$

$$368 \pmod{5} = 3$$

$$368 \pmod{7} = 4$$

$$368 \pmod{11} = 5$$

So the solution is 368.

$$168 = \frac{1155}{3} \pmod{3} \quad (\text{Ans})$$

$$165 = \frac{1155}{7} \pmod{7}$$

$$105 = \frac{1155}{11} \pmod{11}$$

Q8// Derive the CRT based RSA decryption / signing optimization from first principles (computing d_p, d_q and q_{inv}) and demonstrate the $\sim 4\times$ speed up with a fully worked numerical example using small primes p, q .

Ans: In RSA, $n = pq$

where p and q are large primes.

Normally, decryption / signing requires

$$m = c^d \pmod{n}$$

CRT allows this calculation to be performed modulo p and q , which are approximately half the size of n .

Calculate d_p and d_q :

Define, $d_p = d \pmod{p-1}$

and, $d_q = d \pmod{q-1}$

Then calculate,

$$m_p = c^{d_p} \pmod{p}$$

and, $m_q = c^{d_q} \pmod{q}$

Calculate $q^{-1} \pmod{p}$:

Find, $q_{inv} = q^{-1} \pmod{p}$

Recombine:

Calculate,

$$h = q_{inv} (m_p - m_q) \pmod{p}$$

Then, $m = m_q + hq$

This gives, $m = c^d \pmod{n}$

Numerical example:

Take, $p = 11$, $q = 13$

Then, $n = pq = 11 \times 13 = 143$

Let, $e = 7$

Euler's totient:

$$\begin{aligned} \phi(n) &= (11-1)(13-1) \\ &= 10 \times 12 = 120 \end{aligned}$$

Now, $ed \equiv 1 \pmod{120}$

$$\therefore 7d \equiv 1 \pmod{120}$$

Since, $7 \times 103 = 721 = 6(120) + 1$

$$d = 103$$

Suppose ciphertext is $c = 100$

Direct method:

$$m = 100^{103} \pmod{143}$$

This requires modular exponentiation using the full modulus 143

CRT method:

First, $d_p = 103 \pmod{10} = 3$

$$d_q = 103 \pmod{12} = 7$$

Therefore,

$$m_p = 100^3 \pmod{11} = 1$$

$$m_q = 100^7 \pmod{13} = 9$$

Since, $100 \equiv 9 \pmod{13}$,

$$9^2 = 81 \equiv 3$$

$$9^4 \equiv 3^2 = 9$$

So, $9^7 = 9^4 \cdot 9^2 \cdot 9$

$$\equiv 9 \times 3 \times 9 = 243 \equiv 9 \pmod{13}$$

$$\therefore m_q = 9$$

Now we have to find,

$$q^{-1} \pmod{p} = 13^{-1} \pmod{11}$$

P.T.O

Since, $13 \equiv 2 \pmod{11}$

and, $2 \times 6 = 12 \equiv 1 \pmod{11}$

$$q_{inv} = 6$$

$$\therefore h = 6(1-9) \pmod{11}$$

$$= 6(-8) = -48$$

$$\therefore -48 \equiv 7 \pmod{11}$$

therefore, $h = 7$

$$\begin{aligned} \text{Finally } m &= m_q + hq \\ &= 9 + 7(13) \\ &= 9 + 91 \end{aligned}$$

$$\therefore m = 100$$

So, CRT gives the same result as direct RSA decryption.

~4x Speed up Rationale:

The normal computation uses an exponentiation modulo an n -bit number. CRT replaces it with two exponentiations modulo approximately $n/2$ -bit numbers. With ordinary quadratic-cost arithmetic,

$$T(n) \propto n^2$$

$$\text{Therefore, } 2T(n/2) = 2\left(\frac{n}{2}\right)^2 = \frac{n^2}{2}$$

p.T.O

Together with the shorter exponents, this gives an approximately 4x practical speed up for RSA private-key operation.

Q9 Explain the Bellare fault injection attack on RSA-CRT signatures: how a single faulty modular computation during signing leaks a factor of n and what countermeasures (e.g., signature verification before release) prevent it.

Ans: Bellare Fault-Injection Attack on RSA-CRT:

Bellare attack is a fault injection attack against RSA implementation using CRT. The signature computes:

$$s_p = m^d \pmod{p}$$

$$s_q = m^d \pmod{q}$$

And combines them using CRT to obtain the signature s .

Fault injection: An attacker cause an error in one computation then the device produce a faulty signature s .

P.T.O

Suppose a hardware fault corrupts s_1 to s_1' , while s_2 remains correct. The faulty signature s' satisfies:

$$s' \not\equiv s \pmod{p} \Rightarrow (s')^e \not\equiv M \pmod{p}$$

$$s' \equiv s \pmod{q} \Rightarrow (s')^e \equiv M \pmod{q}$$

Therefore $(s')^e - M$ is a multiple of q but not a multiple of p .

$\therefore$ The attacker extracts the private prime factor by computing:

$$q = \gcd((s')^e - M, n)$$

Countermeasures:

- Verify the signature before releasing it.

check: $s^e \equiv m \pmod{n}$

If verification fails do not release the signature.

- Use fault detection/checking during CRT computation.
- Use blinding/randomization techniques.
- Protect the implementation against physical fault injection.

Q10 Compute a modular inverse using both the Extended Euclidean Algorithm and Fermat's Little Theorem-based exponentiation (for a prime modulus) and discuss the trade-offs between the two approaches for large moduli.

Ans: Suppose we want to find,

$$a^{-1} \pmod{p}$$

where, p is prime and $\gcd(a, p) = 1$

Let's calculate, $3^{-1} \pmod{11}$

Method-1: Extended Euclidean Algorithm

We have, $11 = 3(3) + 2$

$$3 = 2(1) + 1$$

$$2 = 1(2) + 0$$

Thus, $\gcd(3, 11) = 1$

Back substitute:

$$\begin{aligned} 1 &= 3 - 2 \\ &= 3 - (11 - 3 \times 3) \\ &= 4(3) - 11 \end{aligned}$$

Therefore, $4(3) \equiv 1 \pmod{11}$

Hence, $3^{-1} \equiv 4 \pmod{11}$

Method-2: Fermat's Little Theorem:

For prime p ,

$$a^{p-1} \equiv 1 \pmod{p}$$

Therefore, $a^{p-2} \equiv a^{-1} \pmod{p}$

For $a=3$, $p=11$:

$$3^{-1} \equiv 3^9 \pmod{11}$$

Now, $3^2 = 9$

$$3^4 = 81 \equiv 4 \pmod{11}$$

$$3^8 \equiv 4^2 = 16 \equiv 5 \pmod{11}$$

Therefore, $3^9 = 3^8 \times 3$

$$\equiv 5 \times 3 = 15 \equiv 4 \pmod{11}$$

Thus, $3^{-1} \equiv 4 \pmod{11}$

Comparison table:

Extended Euclidean Algorithm	Fermat's Little Theorem
① Works for any a, n with $\gcd(a, n) = 1$.	① Requires prime modulus p .
② Directly gives inverse	② Requires exponentiation.
③ Usually efficient for general modular inverse	③ Useful when modulus is prime.
④ Based on Euclidean division.	④ Based on $a^{p-1} \equiv 1 \pmod{p}$

Classical & Symmetric Block ciphers.

Q11 Design and fully execute a playfair cipher encryption and decryption of a phrase of your choosing (minimum 16 letters with a keyword of your choosing) Present the complete 5x5 key matrix as a table and show every digraph substitution step.

Ans: Playfair cipher:

Lets, ^{Plain text} key word: MY NAME IS NAIMA RASA

Key word: TASNIM

Encryption:

Plain text: MY NAME IS NAIMA RASA

Digrams : MY NA ME IS NA IM AR AS AX

T	(A) → S	(N) → I/J		
(M) → B	C	D	E	
F	G	H	K	L
U	P	Q	R	V
V	W	X	(Y)	Z

MY → ~~EX~~ DV

NA → I/J S → IS

~~ME~~

P.T.O

T	A	S	N	I/J
M	B	C	D	E
F	G	H	K	L
O	P	Q	R	U
V	W	X	Y	Z

ME → BM

IS → TNA

NA → I/J S → IS

IM → TE (swap)

AR → NP (swap)

AS → SN

AX → SW (swap)

Encrypted message:

BM IS BM TNA IS TE NP SN SW

Decryption:

Plaintext: BM IS BM TNA IS TE NP SN SW

Digrams: BM IS BM TM IS TE NP SN SW

T	A	S	N	I/J
M	B	C	D	E
F	G	H	K	L
O	P	Q	R	U
V	W	X	Y	Z

BM → ME

IS → NA

BM → ME

TNA → IS

IS → NA

TE → IM

NP → AR

SN → AS

SW → AX

Decriptate code: MYNAME IS NAIMA RASAX

Decriptate message: MY NAME IS NAIMA RASA

Q12 Critically analyze the structural weakness of the Playfair cipher against digraph frequency analysis and explain why it remains vulnerable to know-plaintext attacks despite being far stronger than monoalphabetic substitution.

Ans: Playfair cipher is stronger than a monoalphabetic substitution because it encrypts pairs of letters (digraphs) instead of individual letters. However it still has several structural weaknesses.

They are —

① Digraph frequency analysis: Playfair encrypts two letters at a time but the same plaintext digraph is always transformed according to the same key matrix. Therefore, frequently occurring digraphs can produce recognizable statistical patterns. Example:

Common English pairs such as TH, HE, IN, ER occur frequently. An attacker can analyze

P.T.O

ciphertext digraph frequencies to infer the key structure.

② Limited number of digraphs: With I/J combined, there are only $25 \times 25 = 625$ possible ordered digraphs.

Therefore, statistical analysis is still possible with sufficiently large ciphertext.

③ Structural information leakage: The encryption rules depend on whether two letters are —

- in the same row,
- in the same column or
- at different rows and columns.

Thus, the cipher preserves some structural relationships between letters.

④ Known-plaintext attacks: If an attacker knows some plaintext and its corresponding ciphertext, the attacker obtains information about the positions of letters in the key matrix. Several known plaintext-ciphertext pairs can reveal relationships between matrix positions

and eventually help recover the key.

⑤ Not resistant to modern cryptanalysis:

Playfair has a much larger key space than simple substitution, but it does not provide the strong diffusion and confusion of modern block ciphers.

So we can say that, Playfair is stronger than monoalphabetic substitution but remains vulnerable to statistical and known-plaintext attacks.

Q13 Derive and trace two full rounds of a simplified Feistel-structured cipher (DES like) showing $L_0, R_0 \rightarrow L_1, R_1 \rightarrow L_2, R_2$ symbolically, then with the numeric values $L_0 = 0xAAAAAAAA$, $R_0 = 0xF0F0F0F0$, $K_1 = 0x0F0F0F0F$, $K_2 = 0x33333333$, using XOR as the round function.

Ans: Feistel Round Definition:

A Feistel network divides the input into two halves: (L_i, R_i)

$$L_{i+1} = R_i, \quad R_{i+1} = L_i \oplus F(R_i, K_{i+1})$$

Given, round function $F(R, K) = R \oplus K$. Thus,

$$R_{i+1} = L_i \oplus R_i \oplus K_{i+1}$$

• Round 1:

Given, $L_0 = 0xAAAAAAAA$

$R_0 = 0xF0F0F0F0$

$K_1 = 0x0F0F0F0F$

$$\therefore L_1 = R_0 = 0xF0F0F0F0$$

$$\therefore R_1 = L_0 \oplus R_0 \oplus K_1$$

$$\begin{aligned} &= 0xAAAAAAAA \oplus 0xF0F0F0F0 \oplus 0x0F0F0F0F \\ &= 0xAAAAAAAA \oplus 0xFFFFFFFF \\ &= 0x55555555 \end{aligned}$$

Round-2:

$$K_2 = 0x33333333$$

$$\therefore L_2 = R_1 = 0x55555555$$

$$\therefore R_2 = L_1 \oplus R_1 \oplus K_2$$

$$= 0xF0F0F0F0 \oplus 0x55555555 \oplus 0x33333333$$

$$= 0xF0F0F0F0 \oplus 0x66666666$$

$$= 0x96969696$$

$\therefore$ Final output: $(L_2, R_2) = (0x55555555, 0x96969696)$

Final trace: $(L_0, R_0) \rightarrow (L_1, R_1) \rightarrow (L_2, R_2)$

Q14 // Formally define the avalanche effect in block ciphers. Using a toy 8 bit substitution-permutation network of your own construction, demonstrate how flipping a single input bit changes at least half the output bits and explain why this property is essential for cryptographic security.

Ans: Avalanche effect in a toy 8-bit SPN:

The avalanche effect means that a small change in the plaintext or key should produce a large and unpredictable change in the ciphertext.

P.T.O

Ideally, changing one point input bit should change approximately 50% of the output bits.

For an 8 bit cipher, this means approximately 4 output bits should change when one input bit is flipped.

Toy 8-bit Substitution-Permutation Network:

Divide the 8-bit input into two 4 bit blocks.

Use a simple toy S-box S where:

$$S(0000) = 0000$$

$$S(0001) = 1111$$

The remaining S-box entries can be defined arbitrarily for this toy demonstration.

Use the permutation.

$$P = (1, 5, 2, 6, 3, 7, 4, 8)$$

• Input 1:

Take, $X = 00000000$

Divide: 10000 0000

After substitution: $S(0000)S(0000) = 0000\ 0000$

After permutation: $Y = 00000000$

• Input 2: Flip one bit

Change only the last bit,

$$x' = 00000001$$

Only one input bit change,

Divide: 0000 0001

After substitution: $S(0000)$ $S(0001)$

$$= 00001111$$

After permutation, the four 1-bits are rearranged, but their number remains four.

Thus the output differs in,

4 bits out of 8

$$\therefore \frac{4}{8} = 50\%$$

Therefore, the toy SPN demonstrates the avalanche effect.

Security importance:

Prevents clustering, divide-and-conquer attacks and linear/differential cryptanalysis by making statistical relationships indistinguishable from uniform random noise.

Q15 Evaluate the brute-force resistance of an 8-character alphanumeric password (62-character alphabet) versus a 6 digit PIN, under a modern GPU cracking rate of 10^9 guesses/second. Compute worst case and expected case cracking times for both and discuss the implication for password policy design.

Ans: Give, GPU cracking rate = 10^9 guesses/second.

• 8-character alphanumeric password:

The alphabet contains 62 characters:

$$26 \text{ uppercase} + 26 \text{ lowercase} + 10 \text{ digits} = 62$$

$$\therefore \text{Number of possible passwords} = 62^8 \\ = 2.1834 \times 10^{14}$$

$\therefore$ Worst case time,

$$T_{\max} = \frac{62^8}{10^9} \\ = \frac{2.1834 \times 10^{14}}{1000000000}$$

$$= 218340.1056 \text{ second}$$

$$= \frac{218340.1056}{3600 \times 24} \text{ days}$$

$$= 2.53 \text{ days.}$$

Expected case time:

On average, half the possibilities must be

$$\text{Tried: } T_{\text{avg}} = \frac{T_{\text{max}}}{2} = \frac{2.53}{2} \approx 1.27 \text{ days}$$

• 6-digit PIN:

Each digit has 10 possibilities.

Therefore, $N = 10^6 = 1000000$

$$\begin{aligned} \therefore \text{Worst-case time, } T_{\text{max}} &= \frac{10^6}{10^9} = 10^{-3} \text{ seconds} \\ &= 0.001 \text{ seconds} \\ &= 1 \text{ ms} \end{aligned}$$

Expected case time:

$$T_{\text{avg}} = \frac{0.001}{2} = 0.0005 \text{ seconds} = 0.5 \text{ ms}$$

Implication:

- A 6-digit PIN has a very small search space and is extremely vulnerable to offline brute-force attacks at the stated rate.
- An 8-digit alphanumeric password has a much larger search space but at 10^9 guesses/second it is still far too weak for high-security applications if an attacker obtains the password hash.

Q 16 Compare the error propagation and parallelizability characteristics of ECB, CBC, CFB, OFB and CTR modes using a labelled diagram for each. Recommend the most suitable mode for (a) encrypting a live video stream over a lossy network and (b) encrypting a static disk image, with justification.

Ans: Comparison of AES Modes:

Mode	Parallel Encryption	Parallel Decryption	Bit - flip Error propagation	IV/Nonce.
ECB	Yes	Yes	Only one block	Not required
CBC	Yes No	Yes	Current block + next block	Unique IV
CFB	No	Yes	Current + next block / segment	Unique IV.
OFB	No	No	Only the affected bit.	IV must never repeat
CTR	Yes	Yes	Only the affected bit.	Nonce must never repeat.

P.T.O.

Labelled Diagrams:① ECB:

$$P_1 \rightarrow \text{AES Encrypt} \rightarrow C_1$$

$$P_2 \rightarrow \text{AES Encrypt} \rightarrow C_2$$

$$P_3 \rightarrow \text{AES encryption} \rightarrow C_3$$

② CBC:

$$IV \rightarrow \oplus P_1 \rightarrow \text{AES} \rightarrow C_1$$

$$C_1 \rightarrow \oplus P_2 \rightarrow \text{AES} \rightarrow C_2$$

$$C_2 \rightarrow \oplus P_3 \rightarrow \text{AES} \rightarrow C_3$$

③ CFB:

$$IV \rightarrow \text{AES} \rightarrow \oplus P_1 \rightarrow C_1$$

$$C_1 \rightarrow \text{AES} \rightarrow \oplus P_2 \rightarrow C_2$$

$$C_2 \rightarrow \text{AES} \rightarrow \oplus P_3 \rightarrow C_3$$

④ OFB:

$$IV \rightarrow \text{AES} \rightarrow S_1 \rightarrow \oplus P_1 \rightarrow C_1$$

$$\text{AES} \rightarrow S_2 \rightarrow \oplus P_2 \rightarrow C_2$$

$$\text{AES} \rightarrow S_3 \rightarrow \oplus P_3 \rightarrow C_3$$

⑤ CTR:

$$\text{Nonce} + 1 \rightarrow \text{AES} \rightarrow S_1 \rightarrow \oplus P_1 \rightarrow C_1$$

$$\text{Nonce} + 2 \rightarrow \text{AES} \rightarrow S_2 \rightarrow \oplus P_2 \rightarrow C_2$$

$$\text{Nonce} + 3 \rightarrow \text{AES} \rightarrow S_3 \rightarrow \oplus P_3 \rightarrow C_3$$

P.T.O

Recommendation:

(a) Live video over a lossy network:

CTR is suitable because it supports parallel processing and a bit error affects only the corresponding bit. However, CTR should be combined with authentication in a real system.

(b) Static disk image:

For actual disk encryption, XTS-AES is standard storage mode. Among the five given modes, CTR is preferable because it supports parallel processing and random access.

Q17 Using the partial AES S-box below, perform the subBytes transformation on the state word $[0x23, 0xA7, 0x4c, 0x19]$, then apply shift rows to a 4×4 state matrix that has this word as its first row (fill remaining rows with $0x00$ for the purpose of the exercise). Report the resulting matrix.

Row/col $\rightarrow$	3	4	7	9	A	C
1	6D	1		C6		
2	64					
4		A1				2E
A			63		D2	

Ans: Given that,

Input state word $[0x23, 0xA7, 0x4c, 0x19]$

subBytes using table:

Input byte	High nibble (row)	Low nibble (column)	S-box output
0x23	2	3	D4
0xA7	A	7	63
0x4c	4	C	2E
0x19	1	9	C6

After subBytes: $[D4, 63, 2E, C6]$

Building 4x4 state matrix:

$$\begin{bmatrix} D4 & 63 & 2E & 26 \\ 00 & 00 & 00 & 00 \\ 00 & 00 & 00 & 00 \\ 00 & 00 & 00 & 00 \end{bmatrix}$$

Applying shift rows:

Row 0 → no shift				
Row 1 → shift left by 1				
Row 2 → shift left by 2				
Row 3 → shift left by 3				

no visible effect.

Resulting state matrix:

$$\begin{bmatrix} D4 & 63 & 2E & 26 \\ 00 & 00 & 00 & 00 \\ 00 & 00 & 00 & 00 \\ 00 & 00 & 00 & 00 \end{bmatrix}$$

১৩৩৩

Q18 Derive the AES 128 Rijndael Key schedule and compute the first two round keys (K_1, K_2) from a given 128 bit cipher key of your choosing, showing the RotWord, SubWord, and Rcon steps explicitly.

Ans: Let,

K_0 be the 128 bit all-zero key: $W_0 = W_1 = W_2 = W_3$
 $= 0x00000000$

Round constants: $Rcon[1] = 0x01000000$,

$Rcon[2] = 0x02000000$

AES-Sbox Look up for $0x00$ is $0x63$.

Computing Round key 1 (W_4, W_5, W_6, W_7):

$RotWord(W_3) = RotWord(00000000) = 00000000$

$Subword(00000000) = 63636363$

$$\begin{aligned} W_4 &= W_0 \oplus SubWord(RotWord(W_3)) \oplus Rcon[1] \\ &= 00000000 \oplus 63636363 \oplus 01000000 \\ &= 62636363 \end{aligned}$$

$$W_5 = W_4 \oplus W_1 = 62636363 \oplus 00000000 = 62636363$$

$$W_6 = W_5 \oplus W_2 = 62636363 \oplus 00000000 = 62636363$$

$$W_7 = W_6 \oplus W_3 = 62636363$$

$$\therefore K_1 = 62636363 \quad 62636363 \quad 62636363 \quad 62636363$$

Computing Round key 2 (W_8, W_9, W_{10}, W_{11}):

$$\text{RotWord}(W_7) = \text{RotWord}(62636363) = 63636362$$

$$\text{SubWord}(63636362) = \text{FB FBFBFB (using } S(63) = \text{FB, } S(62) = \text{AA}) \Rightarrow \text{FBFBFBAA}$$

$$W_8 = W_4 \oplus \text{SubWord}(\text{RotWord}(W_7)) \oplus \text{Rcon}[2]$$

$$= 62636363 \oplus \text{FBFBFBAA} \oplus 02000000$$

$$= 9B9898C9$$

$$W_9 = W_8 \oplus W_5 = 9B9898C9 \oplus 62636363$$

$$= F9FBFAAA$$

$$W_{10} = W_9 \oplus W_6 = F9FBFAAA \oplus 62636363 = 9B9899C9$$

$$W_{11} = W_{10} \oplus W_7$$

$$= 9B9899C9 \oplus 63636362$$

$$= F9FBFAAA$$

তাকওয়া

Q19 Explain the Mix Columns transformation as polynomial multiplication over $GF(2^8)$ modulo $x^8 + x^4 + x^3 + x + 1$. Work through the full multiplication of one input column by the fixed Mix Columns matrix, showing the $GF(2^8)$ arithmetic for each byte.

Ans:

Each byte of the AES state is an element of $GF(2^8)$ constructed using the irreducible polynomial,

$$m(x) = x^8 + x^4 + x^3 + x + 1 \quad (\text{hex } 0x11B)$$

Each column of the state (s_0, s_1, s_2, s_3) is then treated as a degree 3-polynomial over $GF(2^8)$

$$S(x) = s_3x^3 + s_2x^2 + s_1x + s_0$$

MixColumns multiplies this by the fixed polynomial

$$a(x) = \{03\}x^3 + \{01\}x^2 + \{01\}x + \{02\}$$

modulo $(x^4 + 1)$ which is equivalent to the matrix

$$\begin{bmatrix} s_0' \\ s_1' \\ s_2' \\ s_3' \end{bmatrix} = \begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \begin{bmatrix} s_0 \\ s_1 \\ s_2 \\ s_3 \end{bmatrix}$$

Worked example:

Taking input column: $s = (d4, bf, 5d, 30)$

$$2 \cdot d4 = \text{time}(d4) = B3$$

$$3 \cdot d4 = 2 \cdot d4 \oplus d4 = B3 \oplus D4 = 67$$

$$2 \cdot bf = \text{time}(bf) = 65$$

$$3 \cdot bf = 65 \oplus BF = DA$$

$$2 \cdot 5d = \text{time}(5d) = BA$$

$$3 \cdot 5d = BA \oplus 5D = E7$$

$$2 \cdot 30 = \text{time}(30) = 60$$

$$3 \cdot 30 = 60 \oplus 30 = 50$$

$$S_0' = 2 \cdot d4 \oplus 3 \cdot bf \oplus 1 \cdot 5d \oplus 1 \cdot 30$$

$$= B3 \oplus DA \oplus 5D \oplus 30$$

$$S_1' = D4 \oplus 65 \oplus E7 \oplus 30 = 66$$

$$S_2' = D4 \oplus BF \oplus BA \oplus 50 = 81$$

$$S_3' = 67 \oplus BF \oplus 5D \oplus 60 = E5$$

$$\therefore \text{output column} = (04, 66, 81, E5)$$

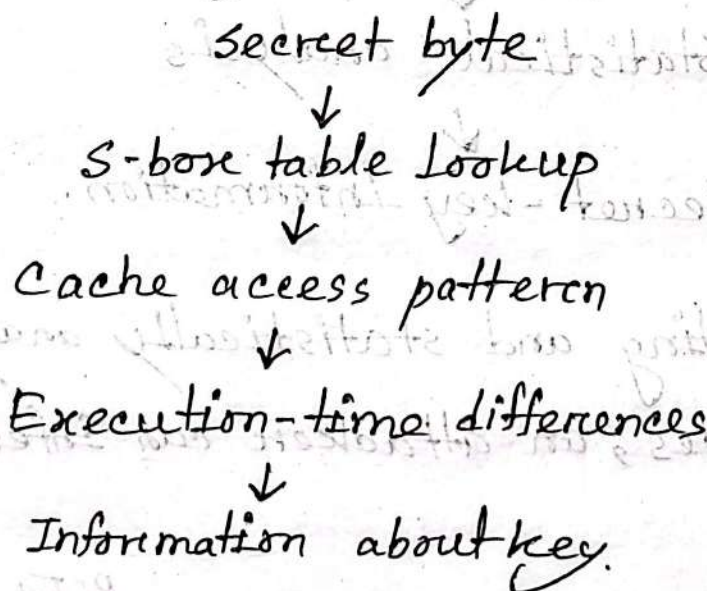
04	66	81	E5
10	20	10	10
80	20	10	10
20	10	10	80

Q 20 Critically evaluate AES's resistance to side-channel attacks, specifically timing analysis (e.g. cache-timing attacks on table-based S-box implementations) and differential power analysis. Describe at least two countermeasures (e.g. constant-time S-box, masking) and their performance trade offs.

Ans:

AES is mathematically strong against classical cryptanalysis, but its implementation can leak information through side channels such as execution time, cache behavior and power consumption.

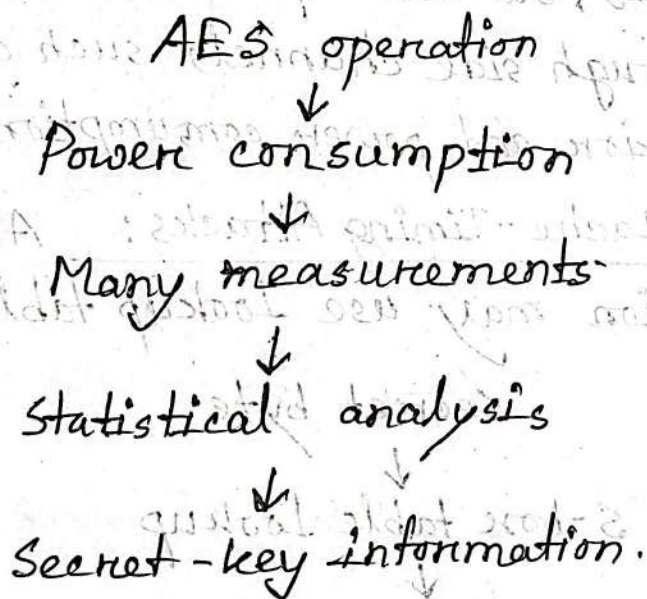
① Timing/Cache-Timing Attacks: A table-based AES implementation may use lookup tables for the S-box.



P.T.O

Table based AES-S box implementations use memory lookups. The lookup pattern depends on secret data, causing different cache behavior and execution times. By observing many timing measurements, an attacker may recover information about the secret key.

② Different Power Analysis (DPA): DPA observes the device's power consumption during AES operations.



By collecting and statistically analyzing many power traces, an attacker can infer secret key bytes.

P.T.O.

Countermeasures:

(a) Constant time / Hardware AES: Use constant-time S-box implementations on hardware AES instructions so execution time does not depend on secret data.

- Advantage $\rightarrow$ Reduces timing / cache leakage.
- Trade off $\rightarrow$ May require more CPU operations on specialized hardware.

(b) Masking: Randomize sensitive intermediate values for example: $[x' = x \oplus r]$

Where (r) is a random mask.

- Advantage $\rightarrow$ Makes power leakage less correlated with the secret.
- Trade-off $\rightarrow$ Requires random numbers and adds computation and implementation complexity.

Therefore, AES itself is secure but careless implementations can be vulnerable to side-channel attacks.

Q 21 || Derive the RSA key-generation encryption and decryption procedures from Euler's Theorem, and prove correctness of decryption (i.e. that $(M^e)^d \equiv M \pmod{n}$) for the general case, including the case $\gcd(M, n) \neq 1$.

Ans: Euler's Theorem: For every positive integer 'a' & 'n' which are said to be relatively prime, then $a^{\phi(n)} \equiv 1 \pmod{n}$

• Key Generation:

Choose two distinct prime numbers p, q
calculate: $n = pq$

Euler's totient:

$$\phi(n) = (p-1)(q-1)$$

Choose a public exponent e such that:

$$1 < e < \phi(n)$$

$$\text{and, } \gcd(e, \phi(n)) = 1$$

Now calculate the private exponent d such that:

$$ed \equiv 1 \pmod{\phi(n)}$$

Therefore, public key: (e, n)

Private key: (d, n) .

• Encryption: Let, the plaintext message be m , where, $0 \leq m < n$.

The ciphertext is calculated as,

$$c = m^e \pmod{n}.$$

• Decryption: The receiver uses the private key d :

$$m' = c^d \pmod{n}$$

Since, $c \equiv m^e \pmod{n}$

We get, $c^d \equiv (m^e)^d$

Therefore, $c^d \equiv m^{ed} \pmod{n}$.

Since, $ed \equiv 1 \pmod{\phi(n)}$

there exists an integer k such that,

$$ed = 1 + k\phi(n)$$

Hence $m^{ed} = m^{1+k\phi(n)}$.

• Correctness when $\gcd(m, n) = 1$.

Euler's theorem states:

$$m^{\phi(n)} \equiv 1 \pmod{n}$$

therefore, $m^{ed} = m^{1+k\phi(n)}$

$$= m(m^{\phi(n)})^k$$

Using Euler's theorem: $m(1)^k$

P.T.O

Therefore, $m^{ed} \equiv m \pmod{n}$

Thus, $e^d \equiv m \pmod{n}$

So, RSA decryption correctly recovers the original message.

• General Case: $\gcd(m, n) \neq 1$

Euler's theorem can not be directly applied when m and n are not relatively prime.

However, RSA still works because:

$$n = pq$$

where p and q are distinct primes.

Consider the congruence modulo p .

• Case 1: $p \nmid m$:

$$\gcd(m, p) = 1$$

By Fermat's theorem:

$$m^{p-1} \equiv 1 \pmod{p}$$

$$\text{Since, } ed = 1 + k(p-1)(q-1)$$

$$\text{we have, } ed \equiv 1 \pmod{p-1}$$

$$\text{Therefore, } m^{ed} \equiv m \pmod{p}$$

P.T.O.

• Case 2: $p \mid m$:

$$m \equiv 0 \pmod{p}$$

therefore,

$$m^{ed} \equiv 0 \equiv m \pmod{p}$$

The same argument applies modulo q .

Thus: $m^{ed} \equiv m \pmod{p}$

and, $m^{ed} \equiv m \pmod{q}$

Since p and q are coprime, by the Chinese Remainder

Theorem (CRT):

$$m^{ed} \equiv m \pmod{pq}$$

Since $n = pq$:

$$m^{ed} \equiv m \pmod{n}$$

Hence RSA decryption is correct even when:

$$\gcd(m, n) \neq 1$$

$$\therefore c^d \equiv m^{ed} \equiv m \pmod{n}$$

Therefore, RSA encryption and decryption are mathematically correct for the general case.

Q 24 Given primes $p=11$, $q=13$, generate a full RSA keypair (choosing a valid public exponent e). Then sign a message hash $H(M)=23$ and verify the signature, showing all intermediate modular exponentiation steps.

Ans: Given,

$$p=11, q=13, H(M)=23$$

key generation:

$$n = 11 \times 13 = 143$$

$$\phi(n) = 10 \times 12 = 120$$

choose:

$$e = 7, \text{gcd}(7, 120) = 1$$

Find d:

$$7d \equiv 1 \pmod{120}$$

$$7(103) \equiv 721 \equiv 6(120) + 1$$

$$\therefore d = 103$$

Therefore, $PU = (7, 143)$

$PR = (103, 143)$

Signature:

$$S = H(M)^d \pmod{n}$$

$$S = 23^{103} \pmod{143}$$

Using repeated squaring:

$$23^2 \equiv 100$$

$$23^4 \equiv 133$$

$$23^8 \equiv 100$$

$$23^{16} \equiv 133$$

$$23^{32} \equiv 100$$

$$23^{64} \equiv 133$$

Since, $103 = 64 + 32 + 4 + 2 + 1$

$$\therefore 23^{103} = 23^{64} \times 23^{32} \times 23^4 \times 23^2 \times 23$$

$$\therefore 23^{103} = (133 \times 100 \times 133 \times 100 \times 23) \bmod 143$$

$$S = 23$$

Verification:

$$S^e \bmod n = 23^7 \bmod 143$$

$$23^7 = 23^4 \times 23^2 \times 23$$

$$\therefore 23^7 \equiv (133 \times 100 \times 23) \bmod 143$$

$$= 23$$

Thus, $S^e \bmod n = H(M)$.

So, the signature is valid.

Q33 Derive a general formula for IPsec ESP overhead percentage in tunnel mode as a function of payload size, header size, authentication-data size and padding, then evaluate it for a 1200 byte payload with an 8-byte ESP header, 16 bytes of authentication data and 2 bytes of padding. Discuss how this overhead scales for small IoT sized packets versus large payloads.

Ans: Given, $L = 1200$ bytes.

Outer IP header = 20 bytes [for tunnel mode IPv4 assumption]

ESP header = 8 bytes

Authentication data = 16 bytes.

ESP Trailer = 2 bytes

Padding = 2 bytes

$\therefore$ Total overhead, $O = 20 + 8 + 16 + 2 + 2 = 48$ bytes.

Taking overhead relative to the original payload:

$$\begin{aligned} \text{Overhead \%} &= \frac{O}{L} \times 100 \\ &= \frac{48}{1200} \times 100 = 4\% \end{aligned}$$

For small IoT packets, the fixed overhead becomes a much larger percentage of the payload. For example with a 20 byte payload:

$$\frac{48}{20} \times 100 = 240\%$$

For large payloads, the overhead percentage becomes smaller.

Q 24 // Execute a full Diffie-Hellman Key exchange with $p=23$, $g=5$, private keys $a=6$ (Aleya) and $b=15$ (Badol), Confirming both parties derive the same shared secret. Then explain the man-in-the-middle vulnerability of unauthenticated DH and describe how an authenticated variant (e.g. the station to station protocol) mitigates it.

Ans: Given, $p=23$, $g=5$

Aleya, $a=6$; Badol, $b=15$

Aleya's public key:

$$A = g^a \cdot \text{mod } p = 5^6 \cdot \text{mod } 23$$

$$5^2 \equiv 2, \quad 5^4 \equiv 4$$

$$A = 4 \times 2 = 8$$

$$\therefore A = 8$$

P.T.O

Badol's public key:

$$B = 5^{15} \bmod 23$$

$$5^2 \equiv 2, 5^4 \equiv 4, 5^8 \equiv 16$$

$$\text{So, } 15 = 8 + 4 + 2 + 1$$

$$B = 16 \times 4 \times 2 \times 5 \bmod 23$$

$$\therefore B = 19$$

Shared secret of Aleya,

$$K_A = B^a \bmod p$$

$$K_A = 19^6 \bmod 23$$

$$19^2 \equiv 16, 19^4 \equiv 3$$

$$K_A = 3 \times 16 = 48 \equiv 2$$

$$\therefore K_A = 2$$

$$K_A = K_B = 2$$

Shared secret of Badol,

$$K_B = A^b \bmod p$$

$$K_B = 8^{15} \bmod 23$$

$$8^2 \equiv 18, 8^4 \equiv 2, 8^8 \equiv 4$$

$$K_B = 4 \times 2 \times 18 \times 8 \bmod 23$$

$$\therefore K_B = 2$$

MITM vulnerability:

Unauthenticated DH does not verify identity. An attacker can replace the public keys and establish separate shared secrets with both parties.

Solution $\rightarrow$ Authenticated DH protocols such as Station-to-Station (STS) use authentication/digital signatures to verify the exchanged DH values and prevent MITM attacks.

Q25 Compare the computational cost of establishing a shared key via RSA key transport versus Diffie-Hellman key agreement at an equivalent security level (e.g. 3072-bit RSA/DH versus 256-bit ECDH.) Discuss why forward secrecy favors (EC)DHE over RSA key transport in modern TLS.

Ans: Comparison of the computational cost of establishing a shared key via RSA key transport vs Diffie-Hellman key agreement at an equivalent security level:

Feature	RSA key transport	DH/ECDH
Basic idea	Client generates a secret and encrypts it with server's RSA public key	Both parties contribute values to derive a shared secret.
Main operation	RSA encryption/decryption.	Modular exponentiation or elliptic-curve scalar multiplication.
Typical classical size	3072 bit RSA	3072-bit DH (256 bit ECDH elliptic curve equivalent)
Static RSA	No, forward secrecy with static RSA.	Yes, with ephemeral DH/ECDH.
Modern TLS use	Legacy approach	Widely used approach.

Forward secrecy:

- **RSA key transport** → The client encrypts the premaster secret with the server's static public key. If the server's private key is compromised years later, an adversary who recorded historical traffic can decrypt all past session.
- **(E)CDHE** → Generates fresh, ephemeral keys per session that are erased from memory immediately after derivation. Long term key compromise does not compromise past session keys.

Computational Cost:

- **3072-bit RSA** → Public operations (small $e = 65537$) are fast, but private key operations require heavy 3072-bit modular exponentiation $O(b^3)$.
- **256-bit ECDH** → Uses scalar point multiplication over a 256 bit prime field; takes $\approx 1/10$ th the compute time and easy use vastly smaller keys (32 bytes vs 384 bytes).

ElGamal Signatures & Elliptic Curve Crypto: Day: / /

Q26 In an ElGamal scheme with $p=467$, $g=2$, private key $x=127$, message hash $H(M)=100$ and nonce $k=213$ (with $\gcd(k, p-1)=1$): compute the public key, generate the signature (r, s) and verify it.

Ans:

Given,

$$p=467, g=2, x=127, H(M)=100, k=213$$

$$\text{where, } \gcd(213, 466)=1$$

• public key:

$$y = g^x \mod p$$

$$= 2^{127} \mod 467$$

Using repeated squaring,

$$2^7 = 128$$

$$2^9 = 512 \equiv 45$$

$$\therefore y = 132 \quad [\text{continuing modular exponentiation}]$$

So the public key is:

$$(p, g, y) = (467, 2, 132)$$

• Signature:

$$\text{First, } r = g^k \mod p$$

$$= 2^{213} \mod 467$$

$$r = 29$$

P.T.O

Now, $s \equiv k^{-1} (H(M) - xr) \pmod{p-1}$

The inverse is,

$$213^{-1} \equiv 431 \pmod{466}$$

and, $H(M) - xr = 100 - (127)(29)$

$$= 100 - 3683 = -3583$$

$$\therefore -3583 \equiv 143 \pmod{466}$$

Therefore,

$$s \equiv 431(143) \pmod{466}$$

$$\therefore s = 51$$

$$\therefore \text{signature } (r, s) = (29, 51)$$

Verification:

ElGamal verification equation:

$$g^{H(M)} \equiv y^r r^s \pmod{p}$$

Left side: $2^{100} \pmod{467} = 189$

Right side: $132^{29} 29^{51} \pmod{467}$

$$132^{29} \pmod{467} = 216$$

$$29^{51} \pmod{467} = 176$$

$$216 \times 176 \pmod{467} = 189$$

$$\therefore \text{Left side} = \text{Right side.}$$

Hence the signature is valid.

Q 27 || Prove algebraically that the ElGamal signature verification equation $g^{H(M)} \equiv y^r r^s \pmod{p}$ holds for a correctly generated signature, starting from the definition $s \equiv k^{-1}(H(M) - xr) \pmod{p-1}$.

Ans: Algebraic proof of correctness:

$$\text{Given } s \equiv k^{-1}(H(M) - xr) \pmod{p-1}$$

Multiply both sides by k :

$$k \cdot s \equiv (H(M) - xr) \pmod{p-1}$$

$$\Rightarrow H(M) \equiv xr + ks \pmod{p-1}$$

By Fermat's Little Theorem,

if exponents are congruent modulo $p-1$, the powers are congruent modulo p :

$$g^{H(M)} \equiv g^{xr + ks} \pmod{p}$$

$$\Rightarrow g^{H(M)} \equiv (g^x)^r \cdot (g^k)^s \pmod{p}$$

Substitute $y = g^x \pmod{p}$ and $r = g^k \pmod{p}$

$$= g^{H(M)} \equiv y^r \cdot r^s \pmod{p}$$

$\therefore$ This completes the proof

Q 28 || On the elliptic curve $y^2 \equiv x^3 + x + 1 \pmod{23}$:
 verify that $P = (3, 10)$ lies on the curve, compute $2P$ via point doubling and compute $P+Q$ for $Q = (9, 7)$, showing the full modular-arithmetic derivation of the slope in the each case.

Ans:

Given that,

$$\text{Curve : } y^2 \equiv x^3 + x + 1 \pmod{23}$$

• Verify $P = (3, 10)$:

$$L.S = y^2 = 10^2 = 100 \equiv 8 \pmod{23}$$

$$R.S = x^3 + x + 1 = 3^3 + 3 + 1 = 31 \equiv 8 \pmod{23}$$

$$\therefore L.S = R.S$$

$\therefore P$ is on the curve.

• Point doubling ($2P$):

$$\text{For, } y^2 \equiv x^3 + ax + b,$$

$$\text{the slope is : } \lambda = \frac{3x_1^2 + a}{2y_1} \pmod{p}$$

Here $a = 1$

$$\therefore \lambda = \frac{3 \cdot 3^2 + 1}{2 \cdot 10} \pmod{23}$$

$$= \frac{28}{20} \equiv \frac{5}{-3} \equiv \frac{5}{20} \equiv \frac{1}{4} \pmod{23}$$

$$\therefore 4^{-1} \pmod{23} = 6 \Rightarrow \lambda = 6$$

$$x_3 = \lambda^2 - 2x_1$$

$$= 6^2 - 2(3) = 36 - 6 = 30 \equiv 7 \pmod{23}$$

$$y_3 = \lambda(x_1 - x_3) - y_1$$

$$= 6(3 - 7) - 10 = -24 - 10 = -34 \equiv 12 \pmod{23}$$

$$\therefore 2P = (7, 12)$$

• Point addition ($P+Q$) for $Q=(9, 7)$:

$$\therefore \lambda = \frac{y_2 - y_1}{x_2 - x_1} \pmod{23}, \quad (7, 9) \neq (3, 3)$$

$$= \frac{7 - 10}{9 - 3} \pmod{23} = \frac{-3}{6} = \frac{-1}{2} \equiv \frac{22}{2} = 11 \pmod{23}$$

$$\therefore x_3 = \lambda^2 - x_1 - x_2 = 11^2 - 3 - 9 = 121 - 12 = 109 \equiv 17 \pmod{23}$$

$$\therefore y_3 = \lambda(x_1 - x_3) - y_1 = 11(3 - 17) - 10 = 11(-14) - 10 = -154 - 10$$

$$\equiv -164 \pmod{23}$$

$$\therefore -164 = -8(23) + 20$$

$$\therefore y_3 = 20$$

$$\therefore P+Q = (17, 20)$$

Q29 On the curve $y^2 \equiv x^3 + 7x + 10 \pmod{37}$ with base point $G = (2, 5)$ of order $n = 19$ and private key $d = 9$; compute the public key $Q = dG$, generate and ECDSA signature (r, s) for $H(m) = 8$ using nonce $k = 3$ and verify it against Q .

Ans:

Given,

$$\text{Curve, } y^2 \equiv x^3 + 7x + 10 \pmod{37}$$

$$G = (2, 5), n = 19 \text{ and } d = 9$$

$$H(m) = 8, k = 3$$

First, verify the given base point $G = (2, 5)$:

$$L.S = y^2 = 5^2 = 25$$

$$\begin{aligned} \text{and, } R.S &= x^3 + 7x + 10 \pmod{37} \\ &= 2^3 + 7(2) + 10 = 8 + 14 + 10 = 32 \pmod{37} \end{aligned}$$

$$\text{since, } 25 \not\equiv 32 \pmod{37}$$

$\therefore G = (2, 5)$ is not on the given elliptic curve.

Therefore G is not a valid base point for this curve. Hence, $Q = 9G, (r, s)$.

and the ECDSA verification can not be correctly

P.T.O

computed from the given parameters.

Q 30 Compare the key size security equivalence between RSA, ElGamal and ECC (e.g. 256 bit ECC $\approx$ 3072-bit RSA/ElGamal at the $\sim$ 128 bit security level and explain why ECC is preferred for resource-constrained devices such as smart cards and IOT sensors.

Ans: Security equivalence table:

Security Level (bits)	RSA/ElGamal Modulus	ECC key Size	Ratio
80	1024 bits	160 bits	$\sim 1:6$
128	3072 bits	256 bits	$\sim 1:12$
256	15360 bits	512 bits	$\sim 1:30$

ECC requires $\longrightarrow$

- Less storage
- Less bandwidth
- Less computation
- Lower power consumption.

Hence ECC is suitable for smart card, IoT sensors and other resource-constrained.

Small key + comparable security $\Rightarrow$ ECC is efficient.

Preference for Resource - Constrained Devices

- **Storage / Bandwidth** → A 256 bit ECC private key requires only 32 bytes of EEPROM vs. 384 bytes for RSA - 3072

- **Power / Compute** → ECC field operations scale as $O(b^2)$ with $b = 256$, whereas RSA scales as $O(b^3)$ with $b = 3072$. ECC provides massive energy savings during sign/key-exchange routines, preserving battery life in

IOT Nodes.

Hash Functions & Lattice-Based (Post-Quantum) Cryptography

Q 31 Formally define preimage resistance, second-preimage resistance and collision resistance for a cryptographic hash function. Derive the birthday-attack complexity ($\approx 2^{n/2}$) for finding a collision in an n bit hash and explain why this motivates 256 bit (rather than 128 bit) output lengths for modern hash functions.

Ans: For a hash function H :

Preimage Resistance: (One way) Computationally infeasible to find M given h such that,
 $H(M) = h$.

Second-Preimage Resistance: (Weak Collision)

Computationally infeasible to find $M_2 \neq M_1$ given M_1 such that, $H(M_1) = H(M_2)$.

Collision Resistance: (Strong Collision) Computationally infeasible to find any pair $M_1 \neq M_2$ such that,
 $H(M_1) = H(M_2)$.

P.T.O

Birthday Attack:

For an n -bit hash, there are " 2^n " possible hash values.

After approximately $\rightarrow 2^{n/2}$ random hash evaluations, the probability of a collision becomes significant. This is the birthday paradox.

Therefore,

- 128-bit hash $\rightarrow$ collision security $\approx 2^{64}$
- 256 bit hash $\rightarrow$ collision security $\approx 2^{128}$

Thus, modern systems prefer 256-bit hash outputs because they provide approximately 128-bit collision security, which is much stronger against collision attack.

Q 32 Trace the Merkle-Damgård construction with a diagram showing message blocks, compression function calls and chaining variables. Explain the length extension attack that this construction is vulnerable to and how HMAC and SHA-3 (Sponge construction) avoid it.

Ans: Merkle-Damgård Construction:

A message is divided into blocks:

$M_1, M_2, \dots, M_t$

The process is:

$IV \xrightarrow{\text{Compression}} H_1$
 $\uparrow M_1$

$H_1 \xrightarrow{\text{Compression}} H_2$
 $\uparrow M_2$

$H_2 \xrightarrow{\text{Compression}} H_3$
 $\uparrow M_3$

...

$H_{t-1} \xrightarrow{\text{Compression}} H_t$
 $\uparrow M_t$

Final Hash.

Mathematically:

$$H_i = f(H_{i-1}, M_i)$$

where, $H_0 = IV$

Length-Extension Attack:

In Merkle-Damgård hashing, the final hash value acts as the internal state.

If an attacker knows: $H(M)$,
They can use it as the starting state and compute: $H(M || \text{padding} || X)$

Without knowing the original message M .

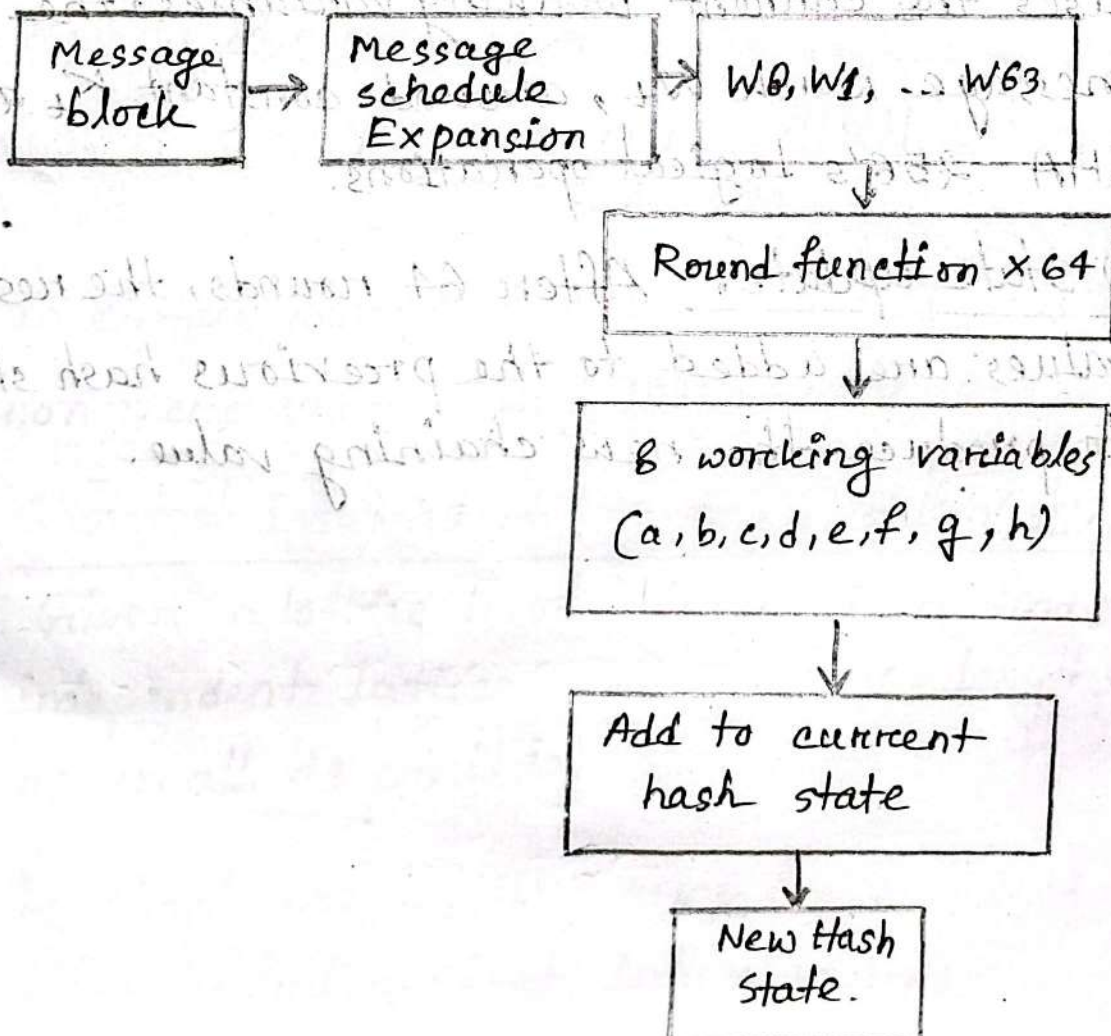
This is called a length-extension attack.

How it is avoid:

- HMAC uses a secret key inside the hash construction, so knowing the outer hash does not allow direct extension of the authenticated message.
- SHA-3 uses a sponge construction, which does not expose the internal state in the same way as Merkle-Damgård, so the traditional length extension attack does not apply.

Q33 || Explain the high level internal structure of the SHA-256 compression function—message schedule expansion, the eight working variables and the round function—using a labelled block diagram. You are not required to reproduce the exact bitwise logic but must correctly identify each stage's role.

Ans: The High Level internal structure of the SHA-256 compression function structure is:



① Message schedule: The 512 bit message block is expanded into 64 words, $W_0, \dots, W_{63}$.

② Working variable: Eight 32-bit variables. $\{a, b, c, d, e, f, g, h\}$ hold the intermediate state during each round.

③ Round function: Each of the 64 rounds uses the current working variables, the message word W_t , around constant K_t and SHA-256's Logical operations.

④ State update: After 64 rounds, the resulting values are added to the previous hash state to produce the new chaining value.

Q 34 Formally define the Shortest Vector Problem (SVP) and the shortest Independent Vector problem (SIVP) in Lattice theory and explain why both are believed to remain hard even for quantum computers (i.e. are not known to be solvable by Shor's algorithm).

Ans:

Shortest Vector Problem (SVP):

Given a lattice L , find a non zero vector $v \in L$ having minimum length:

$$v = \arg \min_{u \in L, u \neq 0} \|u\|$$

In simple words, SVP asks for the shortest non zero vector in a lattice.

Shortest Independent Vectors Problem (SIVP):

Given a lattice L of dimension n , find n linearly independent lattice vectors whose lengths are as small as possible.

In simple words, SIVP asks for a set of short, linearly independent lattice vectors.

P.T. 0

Quantum Security:

SVP and SIVP are believed to remain computationally hard for both classical and quantum computers.

Shor's algorithm efficiently solves problems such as: $\rightarrow$ "integer factoring and discrete logarithm", but no efficient quantum algorithm is currently known for general lattice problems such as SVP/SIVP.

Therefore, lattice-based cryptography is considered a major approach to post-quantum cryptography.

Q35 Explain the core idea behind CRYSTALS - Kyber's Module Learning-With-errors (Module-LWE) key encapsulation mechanism, including how "noise" is used to hide the secret and why this construction resists both classical and quantum attacks.

Ans: CRYSTALS - Kyber / Module-LWE:

CRYSTALS - Kyber is a Lattice-based key encapsulation mechanism (KEM) based on Module-LWE.

The basic idea is : $b = As + e$

where,

$A \rightarrow$ public matrix

$s \rightarrow$ secret vector

$e \rightarrow$ small random noise/error.

$b \rightarrow$ public value.

The noise e makes it difficult to recover the secret s from A and b .

P.T.O

Key idea:

Secret $s + \text{noise } e$



Module-LWE



Public value b .

A legitimate receiver knows the secret and can remove/handle the small noise to recover the shared key.

An attacker sees the noisy equations but solving the underlying Module-LWE problem is computationally difficult.

Therefore, Kyber is designed to resist:

- Classical attacks
- Known quantum attacks.

because no efficient quantum algorithm like Shor's algorithm is known for solving the underlying lattice problem.

Module-LWE + NOISE $\Rightarrow$ Post Quantum Security.

Applied Protocol Security: Kerberos, MAC, TLS & IoT.

Q36// Ans:

Kerberos Authentication Protocol:

Kerberos uses a trusted Authentication Server (AS) and Ticket-Granting server (TGS) for secure authentication.

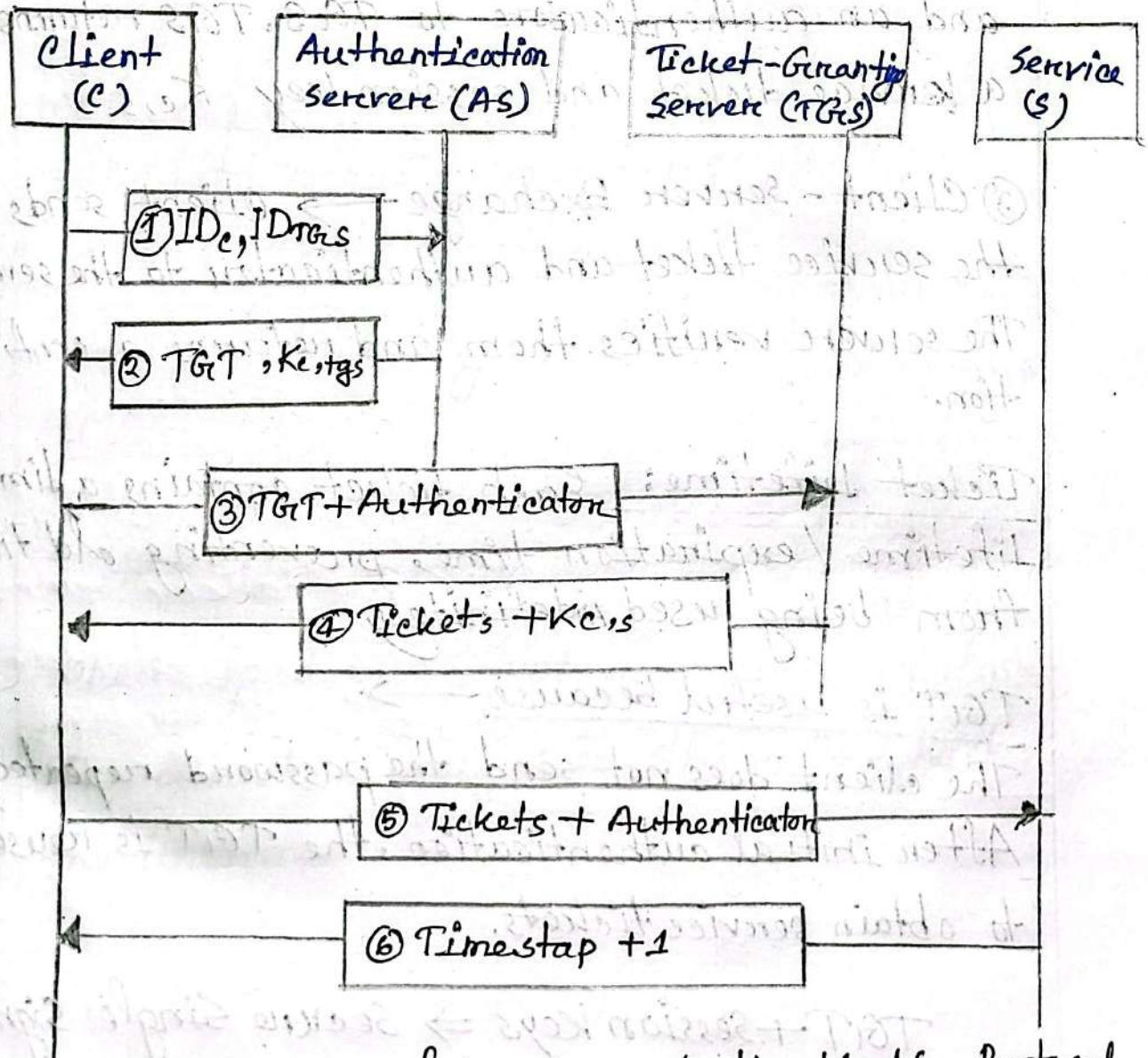


Fig: Diagram of Kerberos Authentication Protocol.

Steps:

① AS Exchange $\rightarrow$ Client requests a Ticket Granting Ticket (TGT). AS sends the TGT and session key $K_{c,as}$, encrypted so that only the client can use the session key.

② TGS Exchange $\rightarrow$ Client sends the TGT and an authenticator to TGS. TGS returns a service ticket and session key $K_{c,s}$.

③ Client - Server Exchange $\rightarrow$ Client sends the service ticket and authentication to the server. The server verifies them and returns a confirmation.

Ticket Lifetime: Each ticket contains a limited lifetime / expiration time, preventing old tickets from being used indefinitely.

TGT is useful because $\rightarrow$

The client does not send the password repeatedly. After initial authentication, the TGT is reused to obtain service tickets.

TGT + Session keys $\Rightarrow$ Secure Single Sign-On.

Q 37 || Ans:

Hash MAC vs HMAC:

A simple MAC may be constructed as,

$$MAC = H(K || M)$$

From Merkle-Damgård hash functions, the hash output can reveal the internal chaining state.

Therefore, an attacker (knowing: $H(K || M)$) may perform a length-extension attack and calculate a valid hash for;

$$K || M || \text{padding} || M'$$

without knowing K .

HMAC:

HMAC uses nested hashing:

$$HMAC(K, M) = H((K \oplus opad) || H((K \oplus ipad) || M))$$

where $ipad$ and $opad$ are fixed constants.

The attacker only sees the outer hash, not the internal state needed to extend the inner message.

So, HMAC prevents the length-extension attack.

P.T.O

Example:

Suppose a simplified hash is:

$$H(x) = \sum x_i \bmod 10$$

Let,

$$K = 3, M = 25$$

A plain construction:

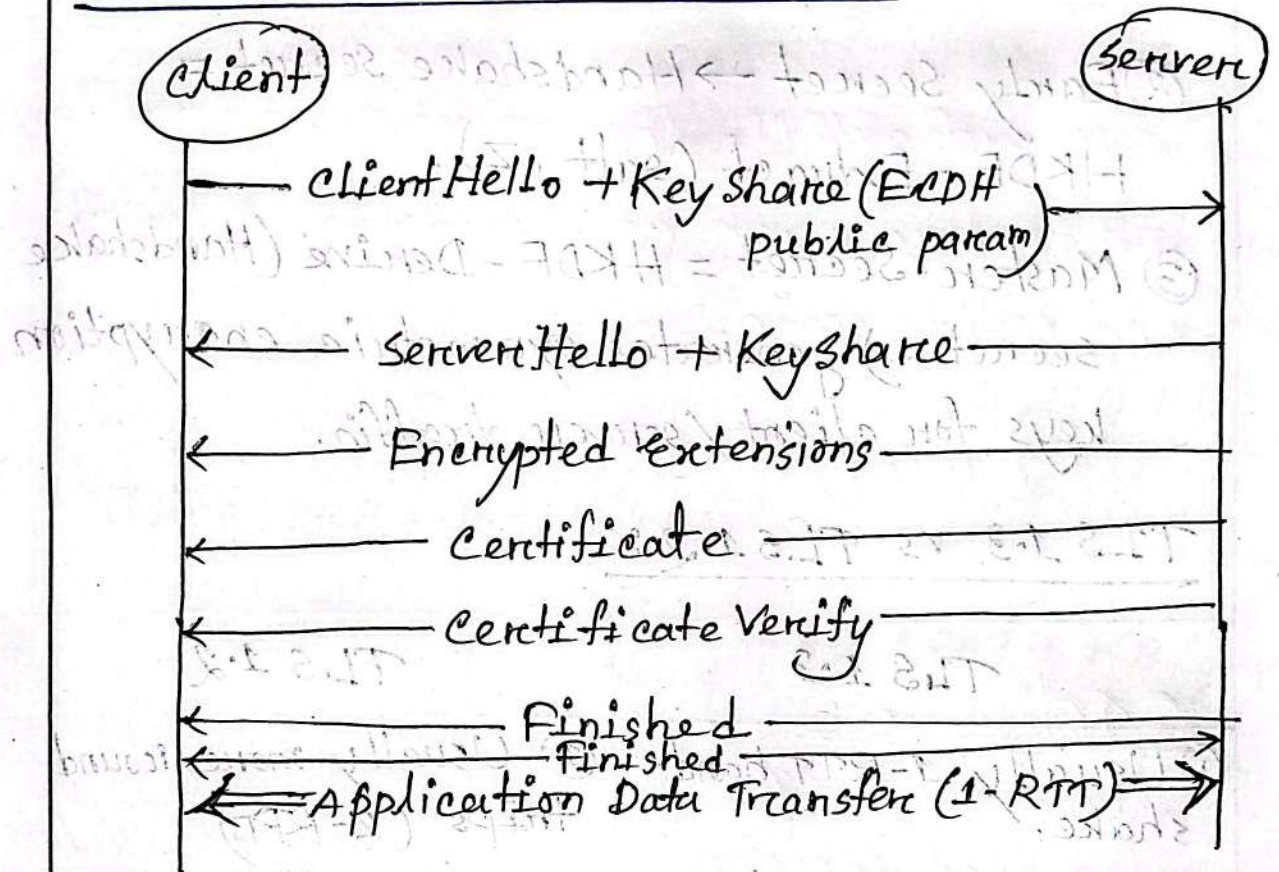
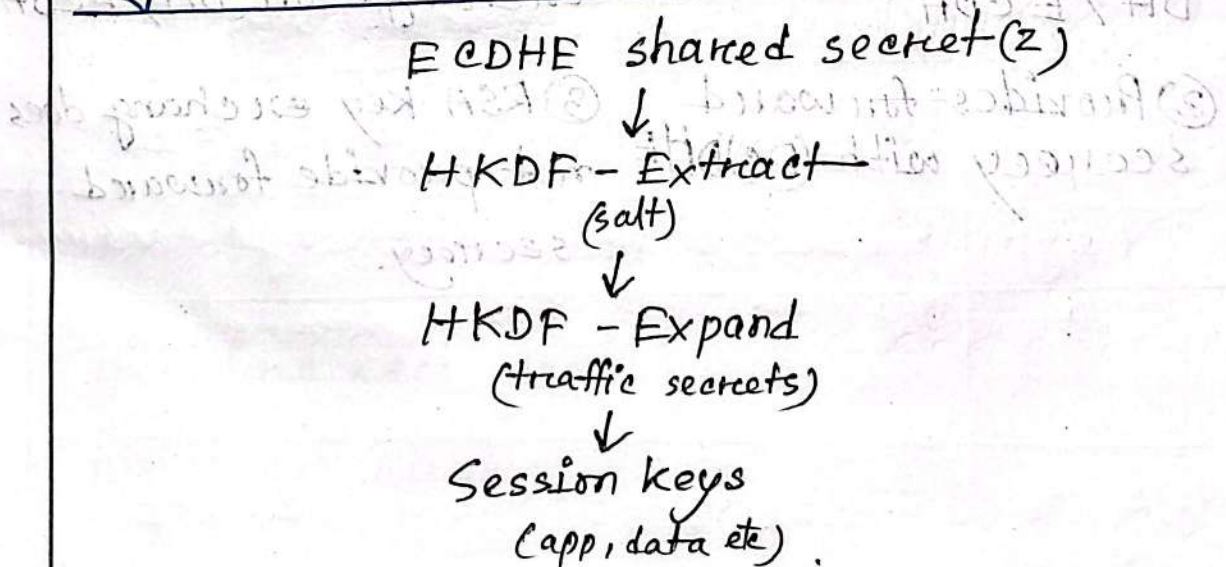
$$H(K || M) = H(325)$$

$$= (3 + 2 + 5) \bmod 10$$

$$= 0$$

The problem is that the exposed hash state can potentially be continued with additional data.

HMAC instead performs an inner hash and then an outer hash, so the attacker cannot simply append data to the exposed MAC.

Q 38// Ans:TLS 1.3 Handshake Sequence:Key Derivation via HKDF:

① Both parties derive the shared secret

$$z = \text{ECDH}(g^x, g^y)$$

② Early Secret $\rightarrow$ Handshake Secret =
HKDF - Extract (salt, z).

③ Master Secret = HKDF - Derive (Handshake secret $\rightarrow$) generates symmetric encryption keys for client/server traffic.

TLS 1.3 vs TLS 1.2 :

TLS 1.3	TLS 1.2
① Usually, 1-RTT hand-shake.	① Usually more round trips (2-RTT)
② Uses ephemeral DH/ECDH	② could use RSA key exchange or DH/ECDH
③ Provides forward secrecy with (E)DHE	③ RSA Key exchange does not provide forward secrecy.

Q39|| Ans:

IoT Bandwidth & Duty-cycle Budget:

Given, 1200 bytes / node / minute

Daily data: $24 \times 60 = 1440$ minutes / day

$$\therefore \text{Daily data} = 1200 \times 1440$$
$$= 1728000 \text{ bytes/day}$$

Using 1 KB = 1024 bytes,

$$\text{It will be } \frac{1728000}{1024} \approx 1687.5 \text{ KB/day}$$
$$\approx 1.69 \text{ MB/day}$$

Duty cycle:

- A 1% duty cycle limit means a device can occupy the channel for at most:

$$0.01 \times 24 \text{ hours}$$

$$= 0.24 \text{ hours}$$

$$= 14.4 \text{ minutes/day}$$

Therefore each 1200 byte transmission must have sufficiently short airtime so that the

P.T.O

total daily airtime remains within the applicable regulatory limit.

For LoRaWAN, the actual airtime depends on factors such as spreading factor bandwidth and coding rate.

$$\text{Duty cycle} = \frac{\text{Total airtime}}{\text{Available time}} \times 100\%$$

Q40 Ans:

Ascon is a lightweight cryptographic standard designed for constrained devices while AES-128 is a widely used general-purpose block cipher. Comparison table-

Parameter	AES-128	Ascon-128
① Design type	Substitution-Permutation Network	Sponge construction/permutation based.
② State size	128 bits	320 bits

Day : _____

Time : _____ Date : / /

Parameter	AES-128	Ascon-128
③ RAM footprint	Moderate (requires S-box tables/256B cache).	Ultra-low (<100 bytes state).
④ Hardware area	~ 2400 GE	~ 1400 GE
⑤ Energy/bit	High on 8-bit/16-bit CPUs	Extremely low (uses 64-bit bitwise ops).
⑥ Side-channel	Prone to cache-timing attacks unless masked	Built in low-overhead masking options.

Recommendation:

Ascon-128 is strongly recommended for a device with <10 KB RAM. Ascon operates using lightweight bitwise logic (AND, XOR, ROTR) that requires no look up tables, avoiding cache-timing leaks and minimizing register and stack usage on constrained microcontrollers.